

Experimental Analysis of In-Memory Hard Cohesion Caching for Write-Intensive Web Applications: Project Design

Simon Gilbu

Department of Computer Science
UiT The Arctic University of Norway
Tromsø, Norway
simon.gilbu@uit.no

Thomas Tekie

Department of Computer Science
UiT The Arctic University of Norway
Tromsø, Norway
thomas.tekie@uit.no

Index Terms—caching, cache invalidation, write-intensive workloads, web applications.

I. INTRODUCTION

This report outlines the planned experimental design for our study on cache invalidation strategies in write heavy web applications[1]. We are looking into whether push based cache invalidation offers benefit over TTL invalidation once write frequency increases, using the model established in our earlier work. The following sections describe the hardware and software environment, workload, evaluation metrics, and experimental protocol we intend to use to answer this question.

II. HARDWARE ENVIROMENT

The experiments are conducted on our personal laptops, with detailed specifications listed in TableI

TABLE I
HARDWARE SPECIFICATIONS

	Laptop 1	Laptop 2
Model	Lenovo IdeaPad 1 15ALC7	Lenovo ThinkPad X390 Yoga
CPU	AMD Ryzen 7 5700U	Intel Core i5-8265U, up to 3.90 GHz
GPU	AMD Radeon (integrated)	Intel UHD Graphics 620
Memory	13.5 GiB	15.4 GiB
OS	Ubuntu 24.04.4 LTS	Kubuntu (x86_64)

III. SOFTWARE ENVIRONMENT

Experiments are implemented in Python. Where automation of repeated runs is required, Python scripts are combined with Bash for orchestration. Git is used for version control of the codebase throughout the project.

IV. WORKLOADS

We will evaluate our different cache scenarios (TTL, No-Cache, PushEvict) against each other using a synthetic simulated workload written in python. Three primary knobs

will be available for the simulation. the request rate and total request count, write and read ratio, and cache keyspace size.

The request rate and request count will be altered both by introducing artificial delays to slow down the requests and by introducing threads to increase the number of generators when a higher throughput is desired.

As per [2] we assume that a write-heavy workload is at least 30% writes, for our workload simulation each request will randomly select to either read or write a key given some adjustable threshold, we aim to simulate writes from 30% and up.

The keyspace will be limited to different ranges in order to provide space bounded caches for our simulation, and will be considered from $keymax = 10\,000$ up to $keymax = 10\,000\,000$, matching the configuration parameters listed in TableII.

We will compare our caching strategies for each configuration of our workloads, We are primarily interested in the difference between caching strategies across the same workload.

For reads we also want to simulate both a normal random distribution and a popularity distribution, since we see from [2] that there in many cases seems to be a preference in access patterns towards Zipfian Distribution[3].

V. DATASET(S)

We will not be relying on external datasets directly (ie, collections of documents, media, traces or external logs), however our workload parameters will be heavily informed by [2]

We find that for this study, synthetic workload generation and evaluation through simulation over that synthetic workload will be appropriate in order to minimize impact of external and/or uncontrolled variables.

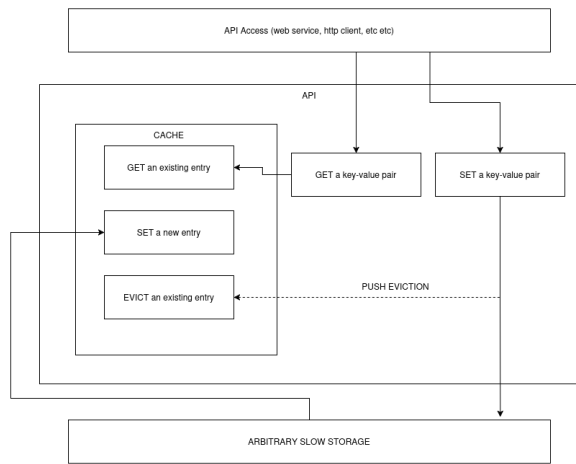


Fig. 1. Model Illustration

VI. MODELS

The evaluation uses a simulation model implemented in python. Three components are represented in the model.

- API Access, representing client access to the underlying storage,
- Arbitrary slow storage, representing underlying storage like a database, network drive or physical media.
- Cache, a key-value store implementing one of our three policies.

We strictly evaluate integer keys and values generated by a workload generator, that is used to access our storage mechanism, this is to abstract away application specific data in order to better evaluate just our three policies against each other.

No-Cache: - Read: Always access backing storage - Write: Directly to backing storage: Reads always observe latest value

TTL-Cache: - Read: Use cached value if unexpired, otherwise fetch and populate - Write: Update backing storage, existing value remains in cache until expired - Cache reads can include stale data

PushEvict: - Read: Use cached value if unexpired; otherwise read and populate from storage: Write: Update backing storage and invalidate cache - First “read” fetches, subsequent reads fetch cached value until evicted.

VII. EVALUATION AND PERFORMANCE METRICS

We evaluate each configuration along two metrics:

[1]

Hit ratio and *miss ratio* measure how often a request is served from the cache versus requiring a fetch from the backend storage. This will show us the benefit of whether it is beneficial or not.

Freshness measures how outdated the data served by the cache is relative to the most recent write to our data source. We are looking at freshness for the cache as a whole, not for the individual data elements, so it is expressed as a ratio rather than a per element measure. It is calculated as the number of fresh hits divided by the total number of hits, where a fresh hit is a cache hit that matches the current value in the storage. This tells us what percentage of hits actually correspond to the correct current data in storage.

VIII. EXPERIMENTAL PROTOCOLS

We are interested in comparing “NoCache”, “TTL”, and “PushEvict” for a handful of different configurations. We intend on doing this by providing the system with some uniform “warmup” work to populate the caches before measuring our metrics over our synthetic workload.

Proposed configurations for each workload and cache-capacity includes every combination of the following values:

TABLE II
EXPERIMENT CONFIGURATION PARAMETERS

Parameter	Values
Read pattern	random, popularity
Keyspace/capacity	10 000, 10 000 000
Write ratio	30%, 50%, 70%, 90%

This gives us a total of $2 * 2 * 4 = 16$ configurations, each of which will be run against each of our cache-implementations multiple times on both available systems and all resulting data will be logged for analysis.

For the TTL Cache, multiple instances will be set up with varying TTL values: 30 ms, 300 ms, 3 000 ms, 30 000 ms and 300 000 ms.

In order to control the pacing of our system, we will execute our GET and SET operations in a threaded environment and regulate the number of requests made per second by introducing an upper bound on acceptable access. Each experiment will run under “slow”, “moderate”, or “high” traffic. In the slow case, we limit access to 10 requests per second. In the moderate case, we limit access to 1 000 requests per second. In the high case, we introduce no upper bound and instead attempt to issue as many requests per second as possible, limited only by the capabilities of the system the experiment is run on.

IX. TIMELINE

Understanding and laying the foundation for our experiment will require gathering the information we need to move forward. We will first finalize the experimental protocol and implement the three cache implementations, then move on to running the first experiments to validate the measurement setup before a full data collection run.

X. DISCUSSION

A. Reproducible Synthetic Workloads

We are considering two different approaches to generating workloads for our experiments. The first is to construct a small set of fixed, hand designed workloads that are reused identically across all configurations, making sure that any observed difference in results comes from the caching strategy rather than the differences in the input.

The second is to randomize the workload for each evaluation run, which reflects the unpredictable reality of the internet traffic but makes it harder to see that differences between configurations and workload variation: since we might have the right configuration but an unfortunate workload for that configuration, or the other way around.

B. Throughput and Reproducibility

Reproducibility may be challenging for the "high" traffic case since we introduce no upper bound on requests per second. The achievable throughput is strongly tied to the hardware the simulation is run on, meaning results from this case may not be directly comparable across our two systems, or reproducible on different hardware.

But this can be overcome by running the simulation on both machines to then cross validate the results. If a pattern such as "PushEvict's hit ratio falls X% faster than TTL's hit ratio at high load" holds consistently across both machines, this would strengthen the finding, even though the absolute throughput numbers themselves are not directly comparable.

REFERENCES

- [1] S. Gilbu and T. Tekie, "Experimental analysis of in-memory hard cohesion caching for write-intensive web applications," 2026, Assignment 1 [2204 / Computer Systems Experiments, Methodologies, Performance Measurements and Evaluation].
- [2] J. Yang, Y. Yue, and K. V. Rashmi, "A large-scale analysis of hundreds of in-memory key-value cache clusters at twitter," *ACM Transactions on Storage*, vol. 17, no. 3, pp. 1–35, Aug. 2021. DOI: 10.1145/3468521
- [3] Wikipedia contributors. "Zipf's law — Wikipedia, the free encyclopedia," Accessed: Sep. 23, 2026. [Online]. Available: https://en.wikipedia.org/wiki/Zipf's_law